

Buscaminas con Backtracking

Documentación Técnica

Inteligencia Artificial

Índice

1. Descripción del Proyecto	2
2. Estructura del Tablero	2
3. Niveles de Dificultad	2
4. Algoritmo de Backtracking para Colocación de Minas	2
5. Flood-Fill Iterativo	3
5.1. Por qué iterativo en lugar de recursivo	3
6. Interfaz Gráfica	3
7. Condición de Victoria	4

1. Descripción del Proyecto

Implementación del clásico juego **Buscaminas** en Java con Swing. Incorpora un algoritmo de **Backtracking** para la colocación de minas y un **flood-fill iterativo** para la expansión de celdas vacías. Incluye múltiples niveles de dificultad, contador de minas y temporizador.

2. Estructura del Tablero

Cada celda del tablero es un objeto con los campos:

- **esMina**: booleano, indica si contiene mina.
- **revelada**: booleano, si el jugador la descubrió.
- **marcada**: booleano, si tiene bandera del jugador.
- **minasVecinas**: entero (0-8), cuenta de minas adyacentes.

3. Niveles de Dificultad

Nivel	Tablero	Minas
Fácil	9×9	10
Intermedio	16×16	40
Difícil	16×30	99

4. Algoritmo de Backtracking para Colocación de Minas

Las minas se colocan aleatoriamente usando backtracking para garantizar que el número solicitado de minas se distribuya correctamente sin superposición:

```
1 void colocarMinas(int totalMinas, int filaSegura,
2                   int colSegura) {
3     int colocadas = 0;
4     Random rand = new Random();
5     while (colocadas < totalMinas) {
6         int f = rand.nextInt(filas);
7         int c = rand.nextInt(cols);
8         // no colocar en celda segura ni en otra mina
9         if (!tablero[f][c].esMina &&
10            !(f == filaSegura && c == colSegura)) {
11             tablero[f][c].esMina = true;
12             colocadas++;
13         }
14     }
15 }
```

Después de colocar las minas, se recorre el tablero para calcular el conteo de minas vecinas para cada celda.

5. Flood-Fill Iterativo

Cuando el jugador revela una celda con **0 minas vecinas**, se expanden automáticamente todas las celdas adyacentes vacías. Esto se implementa con **flood-fill iterativo usando una pila** para evitar `StackOverflowError` en tableros grandes.

```
1 void expandir(int filaInicio, int colInicio) {
2     Stack<int[]> pila = new Stack<>();
3     pila.push(new int[]{filaInicio, colInicio});
4
5     while (!pila.isEmpty()) {
6         int[] celda = pila.pop();
7         int f = celda[0], c = celda[1];
8
9         if (f < 0 || f >= filas || c < 0 || c >= cols)
10             continue;
11         if (tablero[f][c].revelada) continue;
12
13         tablero[f][c].revelada = true;
14
15         if (tablero[f][c].minasVecinas == 0) {
16             // expandir los 8 vecinos
17             for (int df = -1; df <= 1; df++)
18                 for (int dc = -1; dc <= 1; dc++)
19                     if (df != 0 || dc != 0)
20                         pila.push(new int[]{f+df, c+dc});
21         }
22     }
23 }
```

5.1. Por qué iterativo en lugar de recursivo

En tableros grandes (16×30 con 480 celdas), la recursión puede generar cientos de llamadas en la pila del sistema, causando `StackOverflowError`. El enfoque iterativo usa una pila en el *heap*, que tiene capacidad ilimitada práctica.

6. Interfaz Gráfica

- Cada celda es un `JButton` con tamaño fijo de 30×30 píxeles.
- **Clic izquierdo:** revela la celda (dispara flood-fill si es necesario).
- **Clic derecho:** coloca/quita bandera de mina.
- El **contador de minas** y el **temporizador** se actualizan cada segundo con un `javax.swing.Timer`.
- Al descubrir una mina, se revelan todas las minas y el juego termina.

7. Condición de Victoria

El jugador gana cuando todas las celdas sin mina han sido reveladas. Esto se verifica contando celdas reveladas y comparando con `(totalCeldas - totalMinas)`.